Dossier Technique & Manuel Utilisateur

Projet DevOps - Application EtudLife

Verson : v1.2.0

Auteurs :

- Lyna Baouche
- Alicya-Pearl Marras
- Kenza Menad
- Dyhia Sellah

Date : 24/03/2026

📑 Sommaire

1. **Architecture Technique**

2. **Fonctionnalités Détaillées**

4. **Tests effectués**

5. **Guide d'Installation & Déploiement**

1. Architecture Technique

1.1 Stack Technologique

L'application EtudLife repose sur une architecture moderne, conteneurisée et modulaire.

- **Frontend (Client) :** Interface utilisateur développée en HTML5, CSS3 et JavaScript (Vanilla), servie par un serveur Web léger **Nginx** (exposé sur le port 80).
- **Backend (Serveur) :** API REST développée en Java avec le framework **Spring Boot** (exposé sur le port 8080). Il embarque la logique métier, la sécurisation des requêtes et l'ordonnancement des tâches automatiques (Spring Batch).
- **Bases de Données :**
  - **MySQL** (port 3306) : Base de données relationnelle principale stockant les entités durables (Comptes, Offres sauvegardées, Préférences utilisateurs, Événements).
  - **Redis** : Base de données clé-valeur en mémoire utilisée comme système de cache ultra-rapide pour maintenir l'historique et les sessions du Chatbot IA.
- **Services & APIs Externes :**

- **RapidAPI (JSearch)** : Fournisseur de données en temps réel pour l'extraction des offres de stages et d'alternances sur les plateformes professionnelles.
- **Google Gemini API** : Grand Modèle de Langage (LLM) utilisé pour propulser l'agent conversationnel et exploiter la base de connaissances (RAG).

- **DevOps & CI/CD :** Infrastructure gérée via **Docker** et **Docker-compose** pour un déploiement unifié. L'intégration continue est automatisée via **GitHub Actions**.

2. Fonctionnalités Détaillées

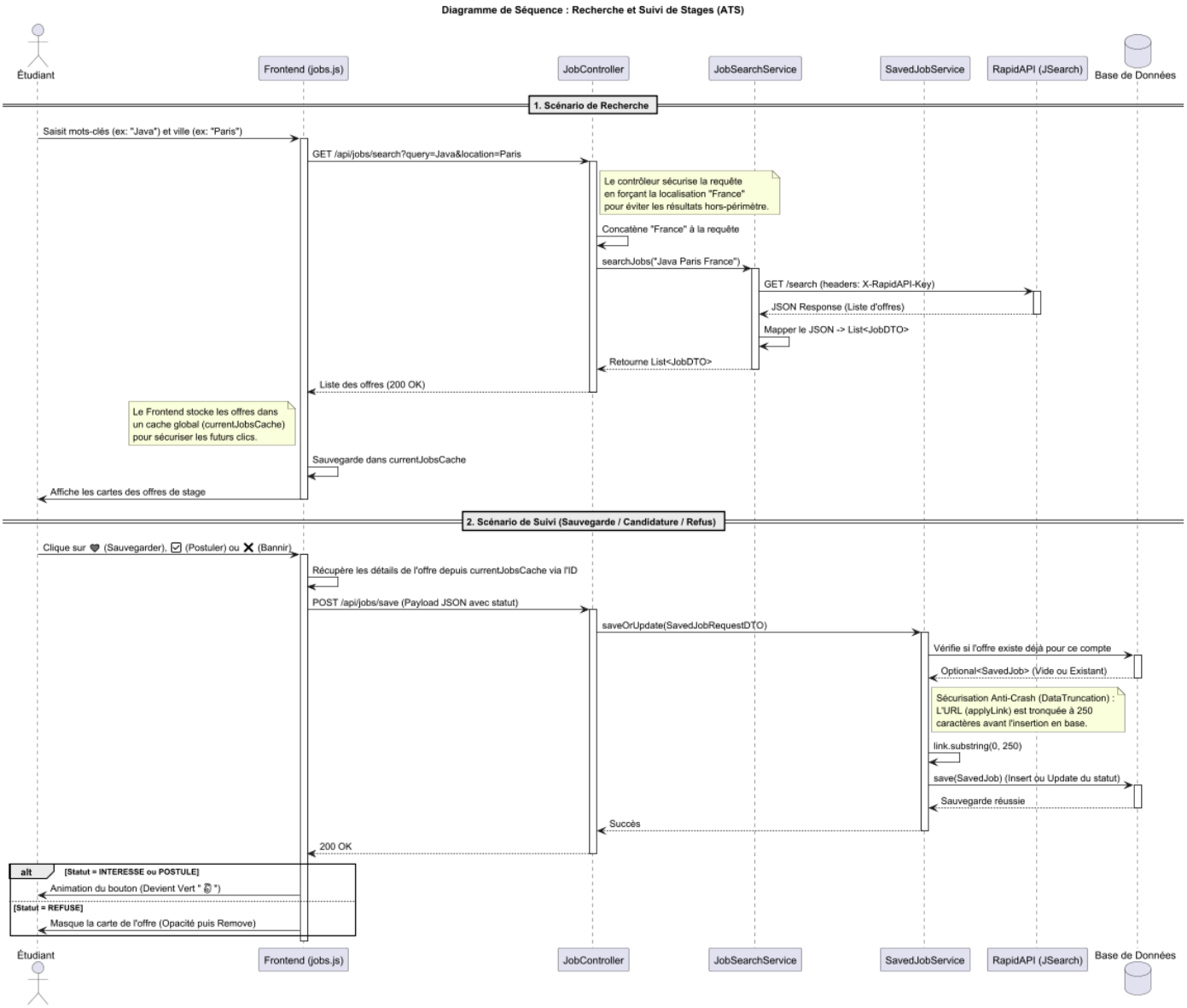
2.1 Chasseur d'offres de stages Feature

2.1.1 Recherche d'offres et suivi des candidatures

Fonctionnalités clés :

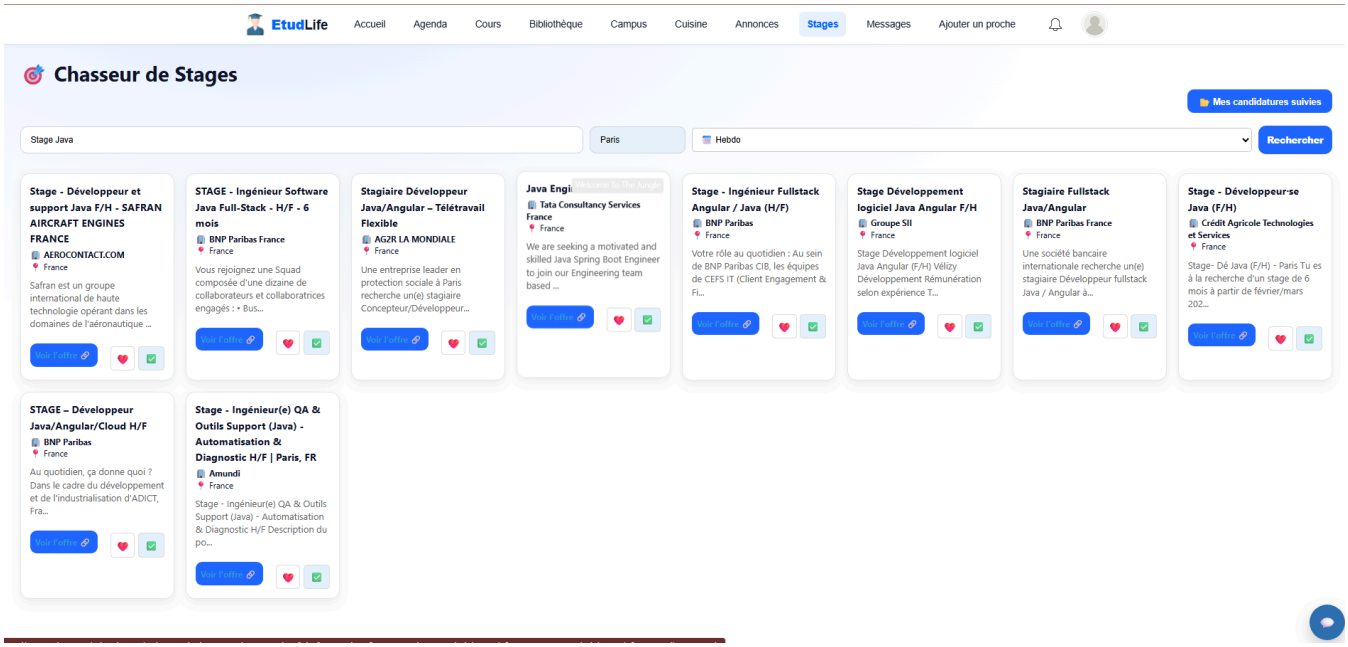
- **Recherche intelligente :** L'utilisateur peut rechercher des offres de stages par mots-clés et localisation. Le backend sécurise la requête en forçant la localisation "France" pour garantir la pertinence des résultats.
- **Gestion des candidatures (ATS) :** L'étudiant peut sauvegarder une offre (statut `INTERESSE`), marquer qu'il a candidaté (statut `POSTULE`), ou masquer une offre non pertinente (statut `REFUSE`).
- **Tableau de bord (KPIs) :** Une page dédiée "Mes candidatures" permet de visualiser les offres sauvegardées, de les filtrer par statut, et d'afficher des indicateurs clés (KPIs) en temps réel (ex: nombre d'offres postulées).
- **Sécurisation des données :** Le système gère dynamiquement la taille des URLs provenant des plateformes externes (LinkedIn, Indeed) pour éviter les erreurs de base de données (Data Truncation).
- **Optimisation des performances via le cache :** Afin de limiter les appels réseau redondants, le module Javascript implémente un système de mise en cache local (`currentJobsCache`). Lorsqu'un utilisateur sauvegarde une offre, le script récupère instantanément les données depuis ce cache mémoire plutôt que de refaire une requête vers le backend.
- **Résilience et Tolérance aux pannes :** Le service d'appel à l'API externe (`JobSearchService`) intègre une gestion stricte des erreurs. En cas d'indisponibilité ou de crash de JSearch, l'application intercepte l'exception et retourne une liste vide à l'utilisateur, empêchant ainsi un crash global du serveur (Erreur 500).
- **Sécurisation et Dédoublonnage :** Le système gère dynamiquement la taille des URLs pour éviter les erreurs de base de données (Data Truncation). Il intègre également une logique de dédoublonnage basée sur l'identifiant externe unique de chaque offre (`externalJobId`), garantissant qu'aucune offre ne s'affiche en double. **Classes Impliquées :**
- `JobController` : Expose les endpoints REST pour la recherche, la sauvegarde et les statistiques.
- `JobSearchService` : Gère la communication HTTP avec l'API externe JSearch.
- `SavedJobService` : Contient la logique métier d'insertion, de mise à jour des statuts et de sécurisation des données.

Implémentation technique

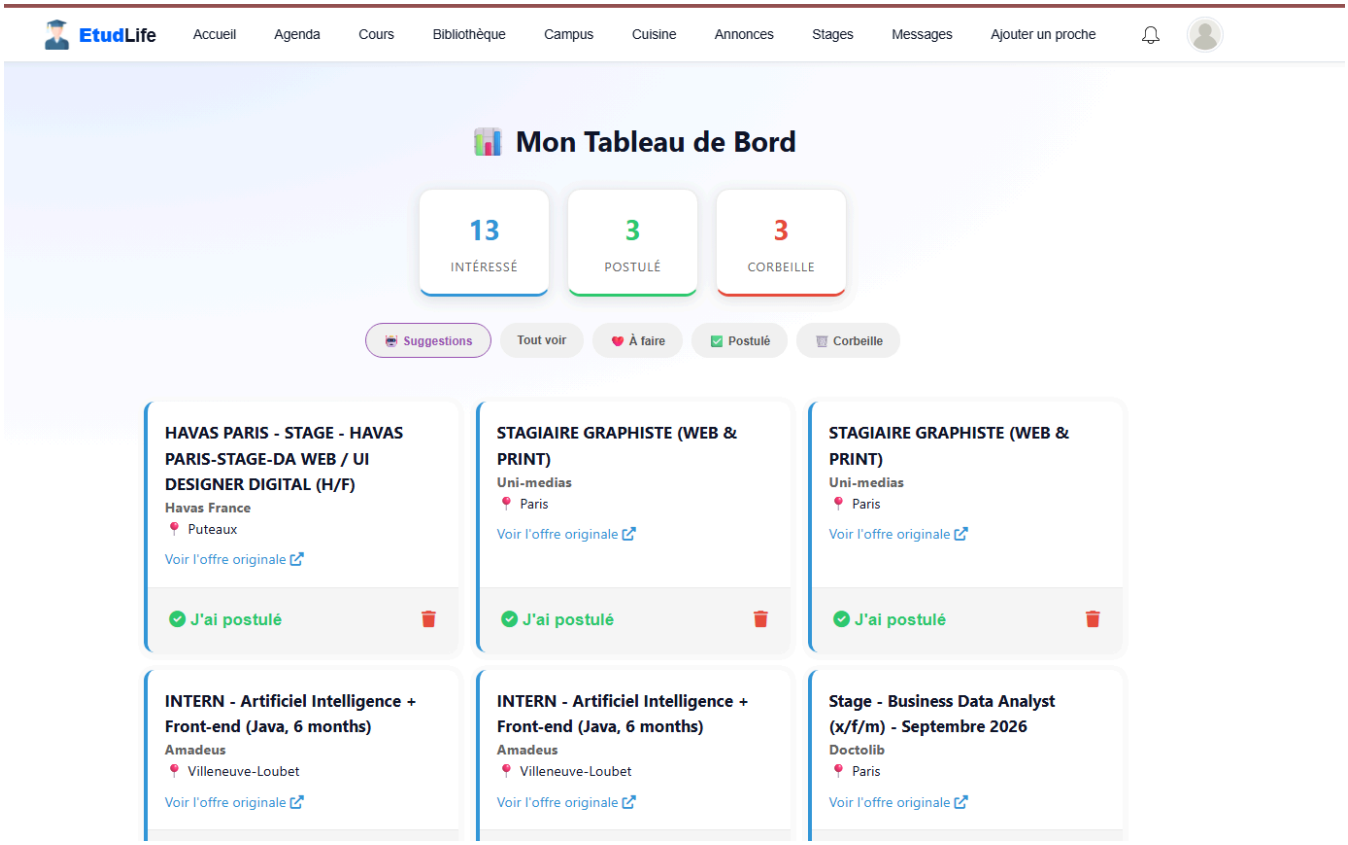


Vue Utilisateur

- L'utilisateur saisit sa recherche d'emplois, il a ensuite la possibilité de sauvegarder ces offres



- Les offres sauvegardées, s'afficheront dans la page du dashboard en cliquand sur 'Mes candidatures suivies'

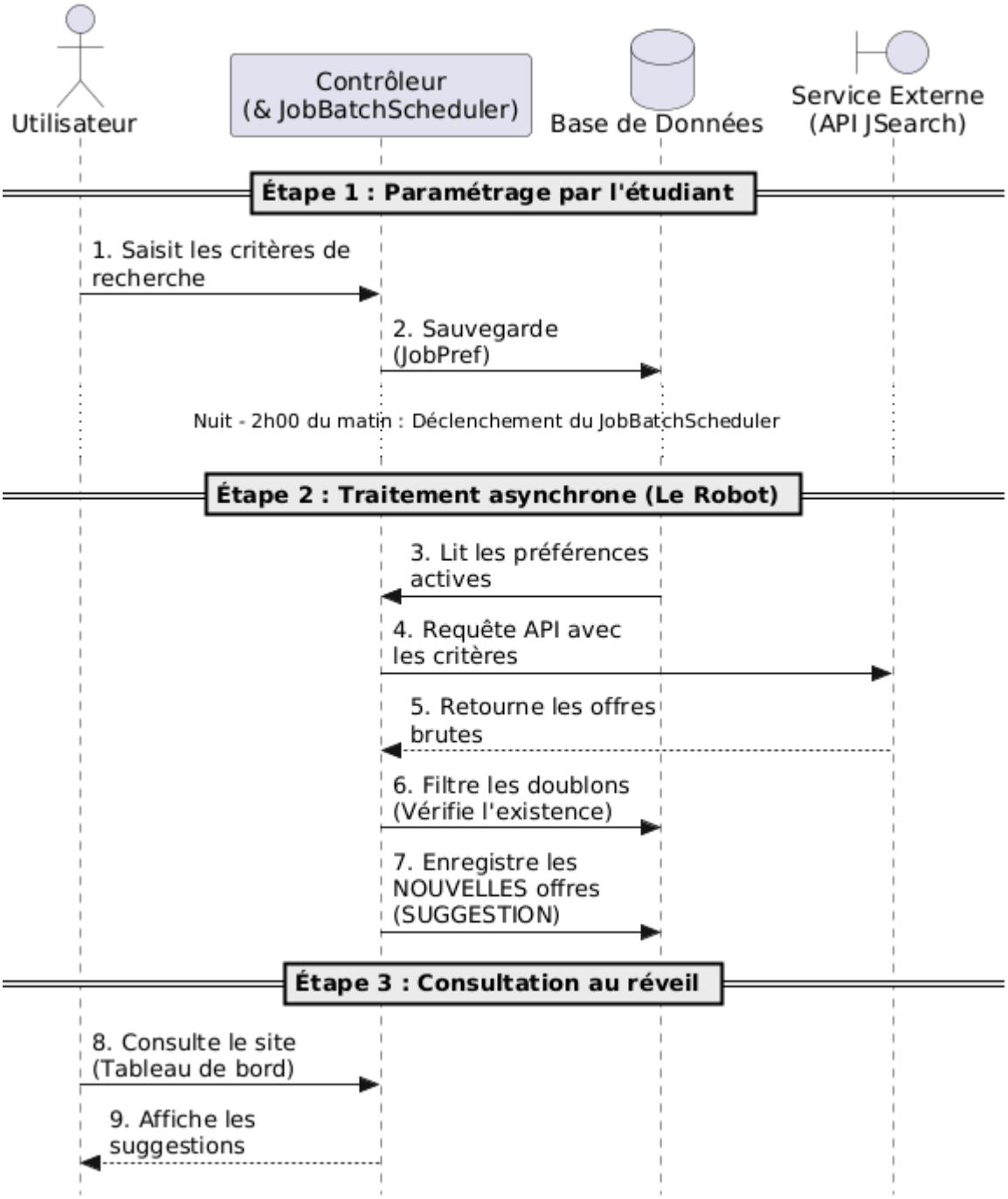


2.1.2 Système de Batch Automatisé pour la recherche d'offres

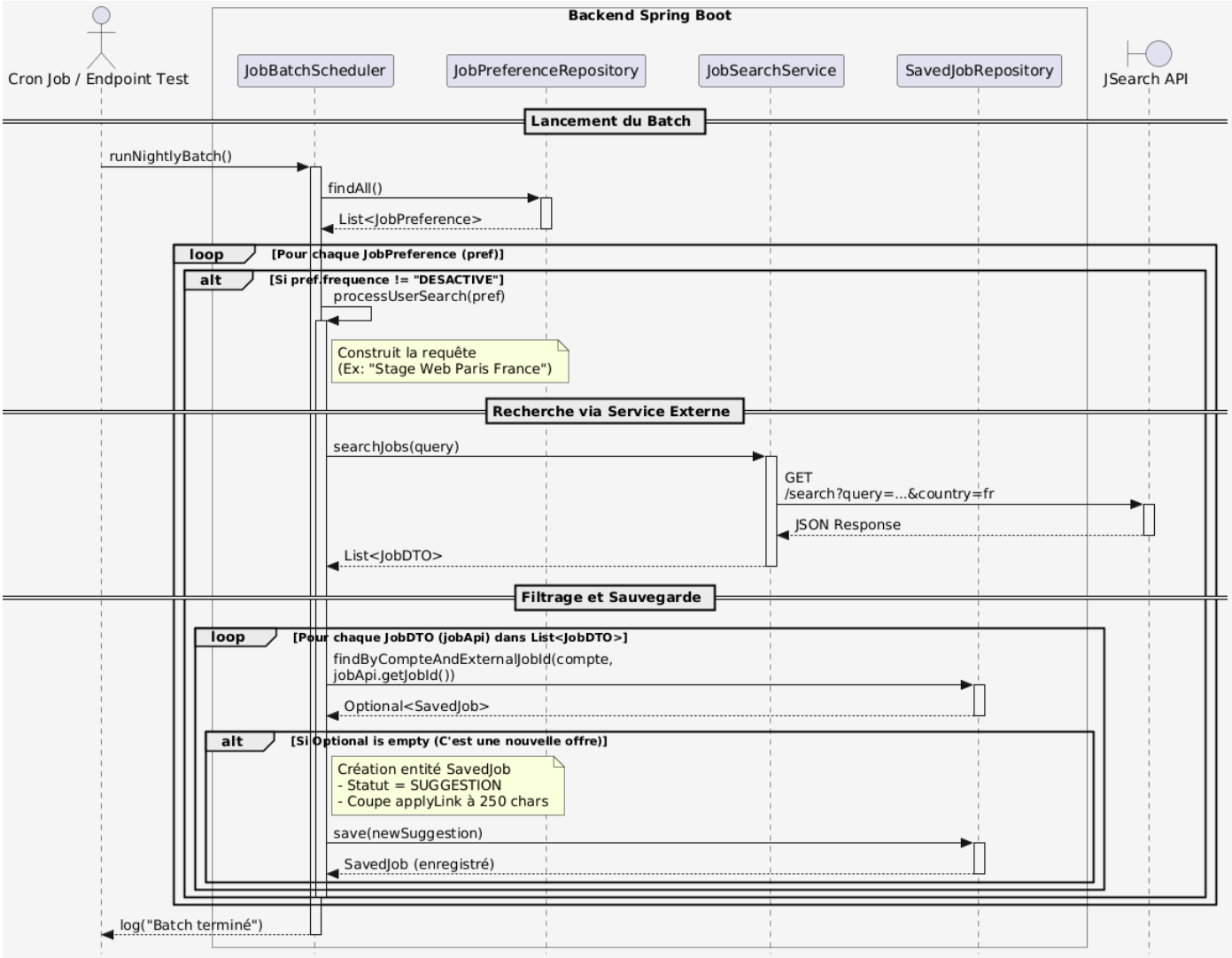
L'idée est simple : on ne veut pas que nos utilisateurs s'épuisent à faire la même recherche tous les jours. L'étudiant enregistre ses critères (ex: "Stage Web à Paris"), et c'est notre serveur qui bosse pour lui la nuit. Le lendemain, il découvre ses nouvelles Suggestions directement sur son tableau de bord.

Par conséquent, nous avons mis en place un job batch automatisé pour exécuter la recherche d'offres de stages chaque nuit à 2h du matin. Ce job utilise le même service `JobSearchService` que les requêtes utilisateur, garantissant ainsi une cohérence totale entre les résultats affichés et ceux récupérés automatiquement.

Logique Métier



Implémentation technique



L'architecture est découpée proprement pour séparer les responsabilités :

- `JobPreference` & `JobPreferenceController` : L'entité qui mémorise ce que cherche l'étudiant (mots-clés, ville) et à quelle fréquence. Le contrôleur s'assure qu'on met à jour ces préférences au lieu de créer des doublons inutiles en base.
- `JobBatchScheduler` : C'est le chef d'orchestre annoté avec @Scheduled. Il boucle sur toutes les préférences actives, prépare les requêtes et filtre les résultats pour ne garder que le neuf.
- `JobSearchService` : Le composant d'intégration externe. Il possède une responsabilité unique : interroger l'API distante JSearch et formater les données brutes reçues en une liste structurée et exploitable d'offres d'emploi.
- `SavedJob` & `JobStatus` : Les offres trouvées par le batch atterrissent ici avec le statut spécial SUGGESTION. Petite astuce technique implémentée : on tronque les URL trop longues (applyLink) à 250 caractères pour éviter que la base de données ne plante lors de l'insertion.

Déploiement Cloud Conteneurisé et Planification Externe

Afin de garantir l'exécution asynchrone du traitement par lots (Batch) dans un environnement Cloud, nous avons conteneurisé notre backend (via **Docker**) et l'avons déployé sur la plateforme **Render**. Cependant, les instances de ce type de service sont sujettes à des mises en veille automatiques (Sleep Mode) après une période d'inactivité. Pour pallier cette contrainte et ne pas dépendre exclusivement du planificateur interne de Spring Boot (@Scheduled), qui s'arrêterait en cas de veille du conteneur, nous avons opté pour une architecture pilotée par des événements externes.

L'approche technique s'articule autour de deux axes :

- Exposition d'un Webhook sur le Conteneur : La logique de déclenchement du Batch a été encapsulée et exposée via une route HTTP spécifique (ex: `/api/test/batch/run`). Cela permet de rendre notre traitement interne accessible et exécutable à la demande, depuis l'extérieur du conteneur.
- Orchestration via un Cron Externe : L'ordonnancement temporel a été délégué à un service tiers spécialisé (tel que **cron-job.org**). Ce dernier est configuré pour émettre une requête HTTP automatisée (un "ping") vers l'endpoint de notre conteneur à une heure fixe (ex: 2h00 du matin).

Résultat : Ce découplage entre la logique métier (embarquée dans le conteneur Spring Boot) et la planification (gérée par le Cron externe) assure une haute résilience. La requête HTTP réveille automatiquement notre conteneur déployé sur Render s'il était inactif, et déclenche le processus d'extraction. Les données sont ainsi systématiquement mises à jour et disponibles pour les étudiants dès leur connexion le lendemain matin.

2.2 Agent IA EtudLife

- L'Agent IA EtudLife transforme l'application en assistant intelligent contextuel pour les étudiants.
- Il repose sur une architecture RAG (Retrieval Augmented Generation).
- Il conserve l'historique des conversations via Redis.
- Il utilise une API externe d'intelligence artificielle fournie par Google (Gemini).
- Il supporte le streaming temps réel via Server-Sent Events (SSE).

Règles Métiers

- **Gestion des sessions** : chaque conversation possède un identifiant unique et expire après 30 minutes d'inactivité.
- **Détection d'intentions simples** : les salutations et remerciements sont traités immédiatement pour fournir une réponse rapide.
- **Détection d'intentions recettes** : les questions liées à la cuisine (budget, ingrédients, recettes) sont interceptées avant le RAG et traitées via RecipeChatService avec génération dynamique via l'API Gemini.
- **Modes de question** : les questions sont classées en AUTO, SITE ou REGLEMENT selon leur nature.
- **RAG documentaire** : le chatbot répond uniquement à partir des documents disponibles, garantissant des réponses fiables et contextualisées.
- **Streaming interactif** : les réponses longues peuvent être envoyées progressivement pour améliorer l'expérience utilisateur.
- **Intégration des offres d'emploi** : le chatbot détecte les intentions liées aux offres sauvegardées et retourne directement les données depuis la base sans passer par Gemini ni le RAG.
- **Détection d'intentions emploi** : les questions contenant des mots-clés liés aux offres ("liste mes offres", "mes candidatures", "j'ai postulé", etc.) sont interceptées avant le RAG et traitées via le service dédié.
- **Statuts gérés** : INTERESSE (offres marquées intéressantes) et POSTULE (offres pour lesquelles l'étudiant a postulé).
- **Dédoublonnage** : les offres sont dédoublonnées par externalJobId avant d'être retournées.
- **Format de réponse structuré** : les offres sont encodées au format JOB_TITLE: / JOB_ITEM:titre|localisation|lien pour permettre un rendu en cartes cliquables côté frontend.

Classes Impliquées

- `ChatController` endpoints REST pour créer une session, envoyer un message, consulter l'historique et fermer une session.
- `ChatStreamController` endpoint REST pour le streaming des réponses en temps réel.
- `ChatService` logique métier principale, traitement des questions et génération des réponses.
- `ChatSessionService` gestion des sessions et de l'historique conversationnel.
- `ChatStreamService` gestion du découpage et de l'envoi progressif des réponses pour le streaming.
- `GeminiClient` appel à l'API externe Gemini pour générer les réponses.
- `PdfKnowledgeBase` recherche des extraits pertinents dans la base documentaire.

- `SavedJobService` récupération et dédoublonnage des offres sauvegardées par statut pour un compte donné.
- `JobSearchService` appel à l'API externe JSearch (RapidAPI) pour la recherche d'offres LinkedIn/Indeed.
- `RecipeChatService` détection des questions liées aux recettes et génération de réponses dynamiques via l'API Gemini.
- `SavedJob` entité représentant une offre sauvegardée avec son statut (INTERESSE, POSTULE, REFUSE, SUGGESTION).

Algorithme & Logique Backend :

Le chatbot expose plusieurs endpoints REST permettant de créer une session, envoyer un message, consulter l'historique, fermer une session ou activer le streaming temps réel.

Lorsqu'un message est reçu, les données sont validées. Si nécessaire, une nouvelle session est générée et stockée dans Redis avec un TTL de 30 minutes. Le message est ensuite enregistré afin de conserver l'historique conversationnel.

Le système intercepte d'abord certaines intentions simples (salutations, remerciements) pour éviter un appel inutile à l'API externe. Le mode de fonctionnement (AUTO, SITE, REGLEMENT) est ensuite déterminé, puis une recherche est effectuée dans la base documentaire via le mécanisme RAG.

Si aucun contenu pertinent n'est trouvé, une réponse standardisée est renvoyée afin d'éviter toute hallucination. Sinon, un prompt structuré est construit et envoyé à l'API Gemini. La réponse générée est sauvegardée dans Redis puis retournée au frontend avec les sources associées.

En mode streaming, la réponse est découpée et envoyée progressivement via SSE afin d'améliorer l'expérience utilisateur.

Avant d'effectuer la recherche RAG, le système vérifie si la question correspond à une intention liée aux offres d'emploi sauvegardées. Si c'est le cas, les offres sont récupérées depuis la base de données selon leur statut (INTERESSE ou POSTULE), dédoublonnées par identifiant externe, puis encodées dans un format structuré (JOB_ITEM). Ce format est interprété par le frontend pour afficher des cartes cliquables avec titre, localisation et lien de candidature, sans aucun appel à Gemini.

2.2.3 Extension : recettes via IA

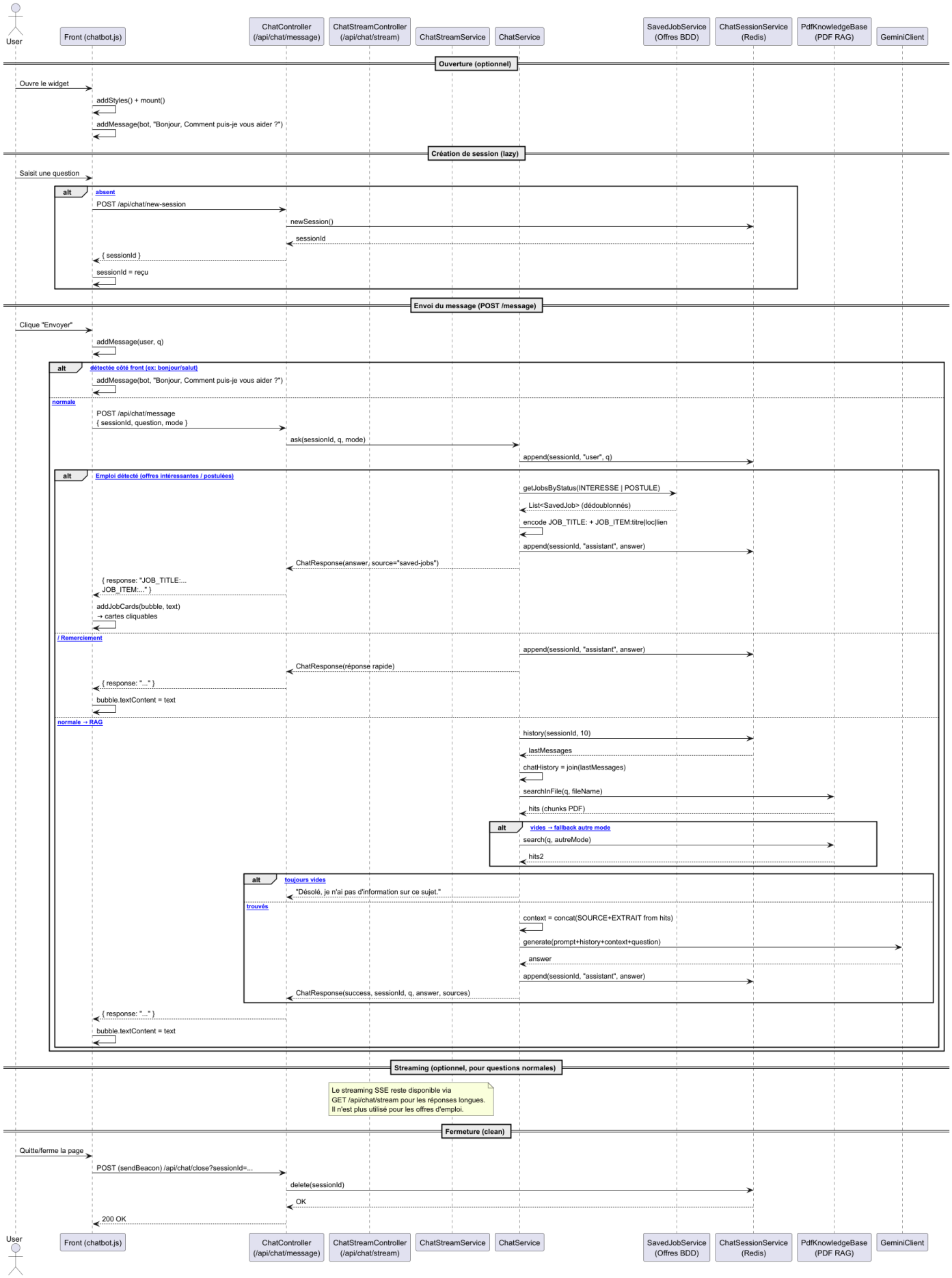
Une fonctionnalité de génération de recettes a été ajoutée au chatbot.

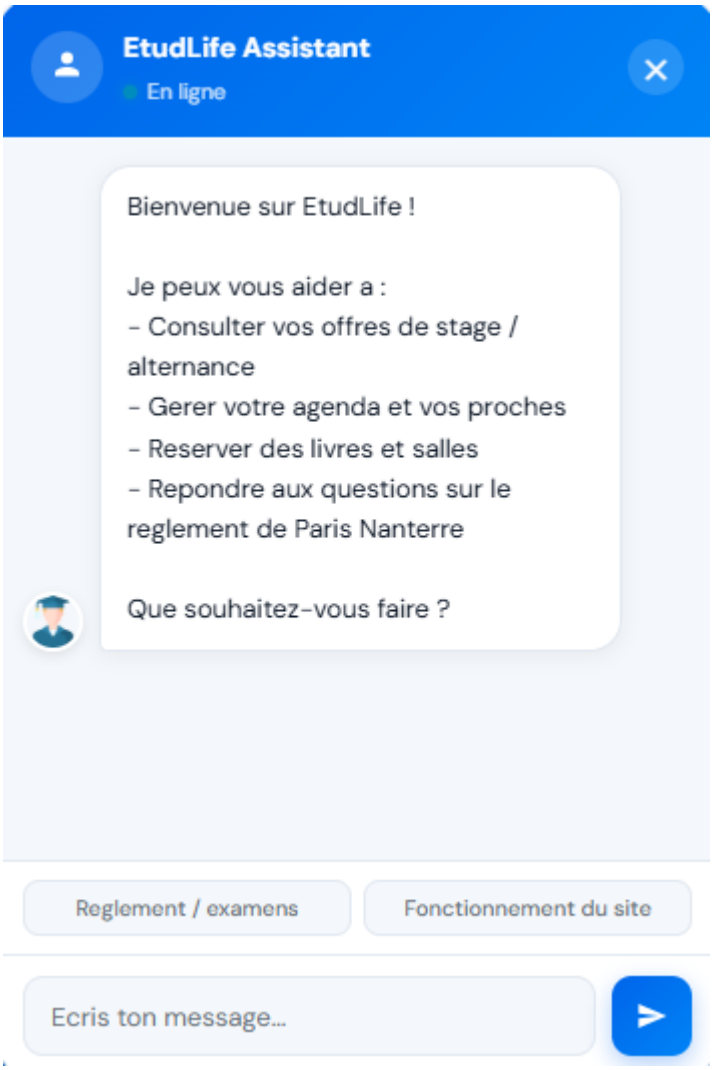
Les questions liées à la cuisine (budget, ingrédients, recettes) sont détectées et traitées via `RecipeChatService` , qui génère une réponse dynamique grâce à l'API Gemini.

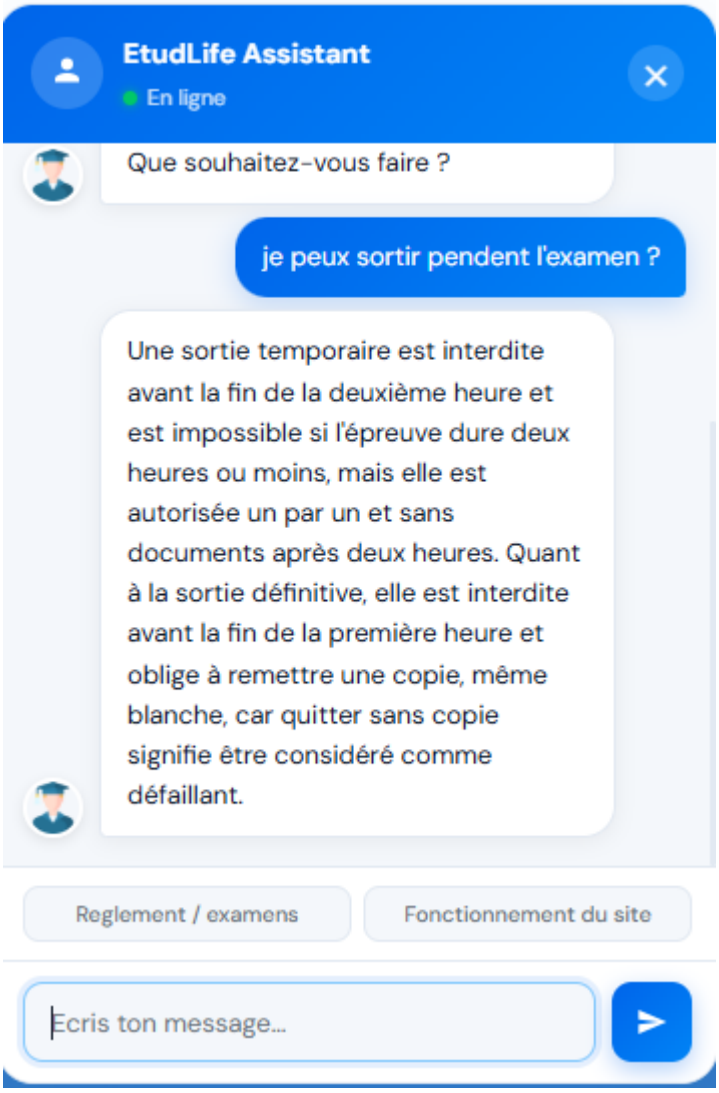
Les réponses sont identifiées par la source "recipes-ai". Elle permet à l'utilisateur de :

demander une recette selon un budget, proposer une recette à partir d'ingrédients disponibles, obtenir des idées de repas simples et rapides

EtudLife Assistant (Agent IA) — Diagramme de séquence

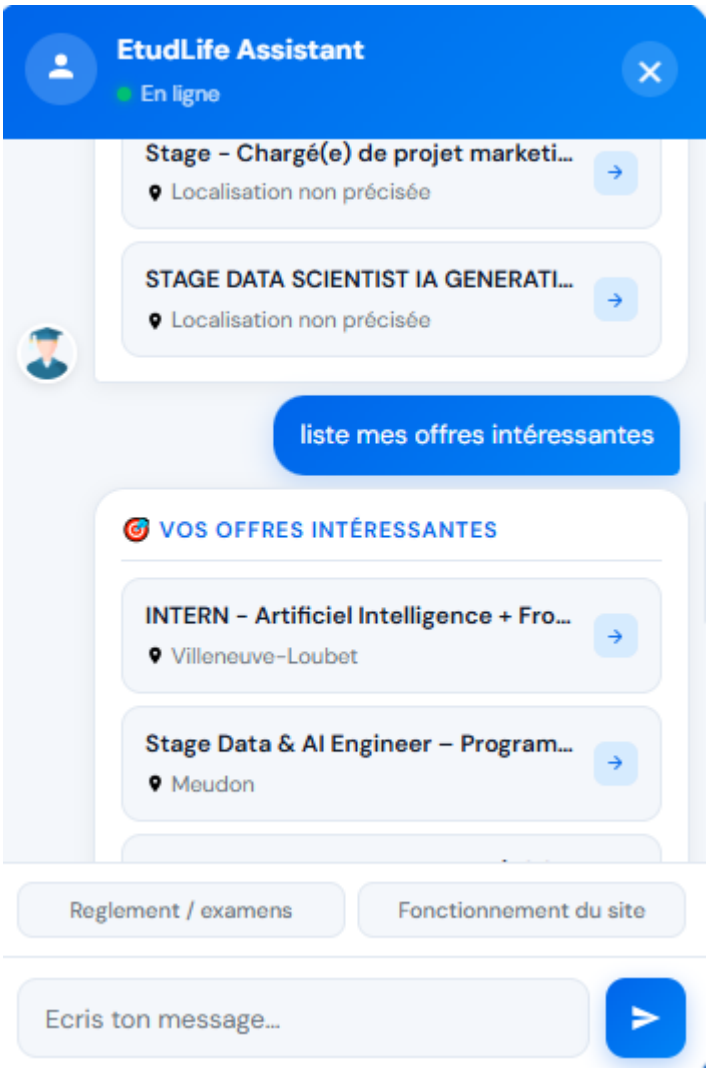


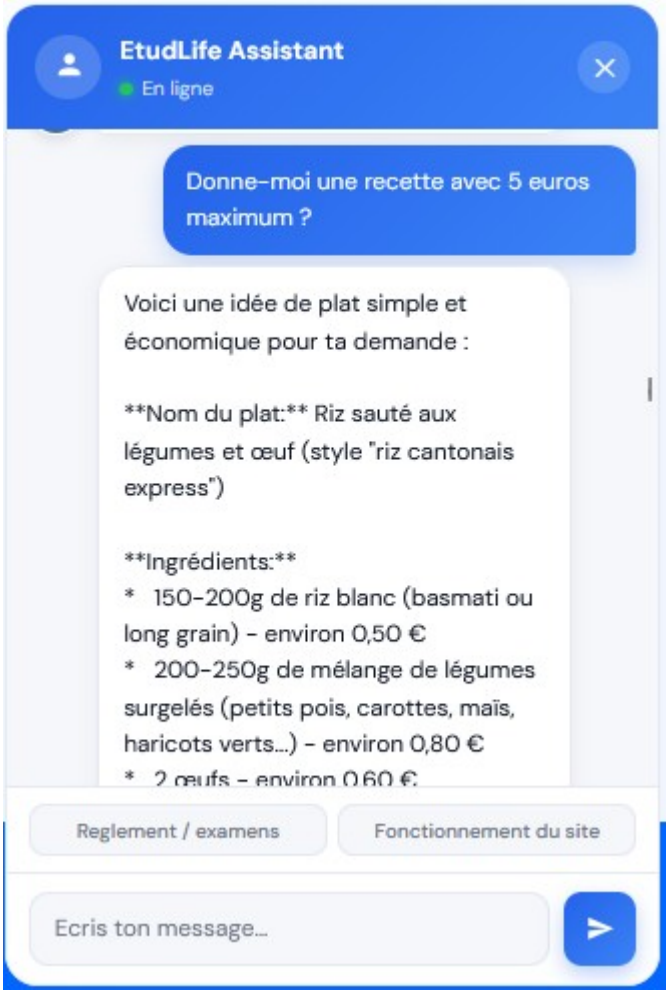

EtudLife Assistant
En ligne
Bienvenue sur EtudLife !
Je peux vous aider a :
- Consulter vos offres de stage / alternance
- Gerer votre agenda et vos proches
- Reserver des livres et salles
- Repondre aux questions sur le reglement de Paris Nanterre
Que souhaitez-vous faire ?
Reglement / examens
Fonctionnement du site
Ecris ton message...

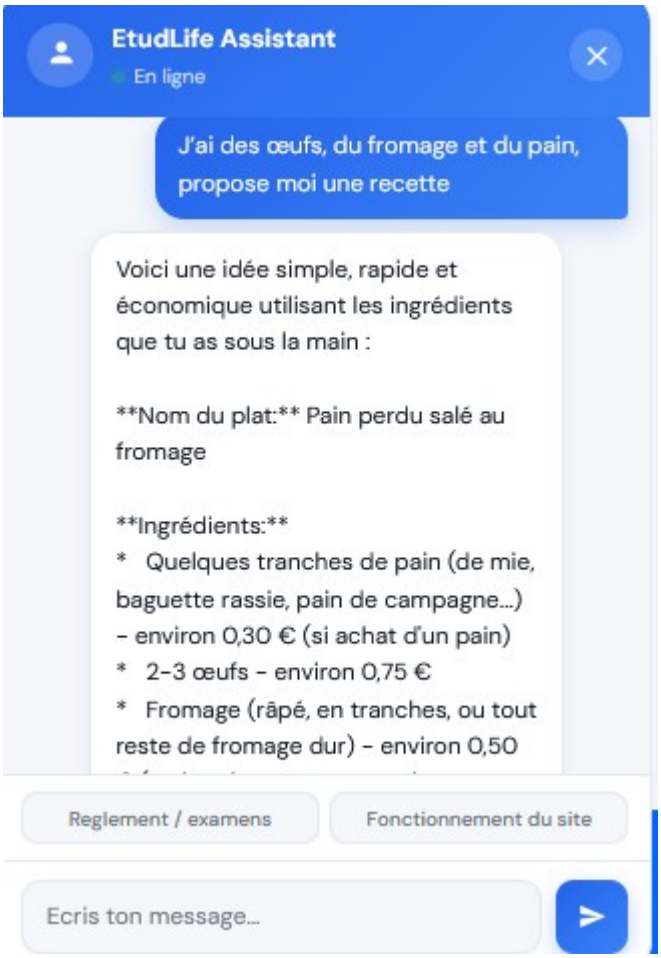

EtudLife Assistant
En ligne
Que souhaitez-vous faire ?
je peux sortir pendent l'examen ?
Une sortie temporaire est interdite avant la fin de la deuxième heure et est impossible si l'épreuve dure deux heures ou moins, mais elle est autorisée un par un et sans documents après deux heures. Quant à la sortie définitive, elle est interdite avant la fin de la première heure et oblige à remettre une copie, même blanche, car quitter sans copie signifie être considéré comme défaillant.
Reglement / examens
Fonctionnement du site
Ecris ton message...

EtudLife Assistant
En ligne
défaillant.
comment créer une annonce?
Pour créer une annonce afin de publier une offre comme un logement, des cours ou la vente d'objets, il faut aller dans le menu "Annonces", puis cliquer sur le bouton "Créer une annonce". Vous devrez ensuite remplir un formulaire en y indiquant le titre, la catégorie, le prix, la localisation et une description détaillée, avec la possibilité d'ajouter une photo. Enfin, cliquez sur "Publier" pour mettre l'annonce en ligne.
Reglement / examens
Fonctionnement du site
Ecris ton message...

EtudLife Assistant
En ligne
l'annonce en ligne.
montre mes offres postulées
VOS OFFRES POUR LESQUELLES VOUS AVEZ POSTULÉ
Stagiaire Consultant(e) Data Engine...
Paris
Stage - Chargé(e) de projet marketi...
Localisation non précisée
STAGE DATA SCIENTIST IA GENERATI...
Localisation non précisée
Reglement / examens
Fonctionnement du site
Ecris ton message...


EtudLife Assistant
En ligne
Stage - Chargé(e) de projet marketi...
Localisation non précisée
STAGE DATA SCIENTIST IA GENERATI...
Localisation non précisée
liste mes offres intéressantes
VOS OFFRES INTÉRESSANTES
INTERN - Artificiel Intelligence + Fro...
Villeneuve-Loubet
Stage Data & AI Engineer – Program...
Meudon
Reglement / examens
Fonctionnement du site
Ecris ton message...


EtudLife Assistant
En ligne
Donne-moi une recette avec 5 euros maximum ?
Voici une idée de plat simple et économique pour ta demande :
Nom du plat: Riz sauté aux légumes et œuf (style "riz cantonais express")
Ingrédients:
* 150-200g de riz blanc (basmati ou long grain) - environ 0,50 €
* 200-250g de mélange de légumes surgelés (petits pois, carottes, maïs, haricots verts...) - environ 0,80 €
* 2 œufs - environ 0,60 €
Reglement / examens
Fonctionnement du site
Ecris ton message...



4. Tests effectués

4.1 Tests du module "Chasseur de Stages"

4.1.1 Recherche de stages (JobSearch)

Pour garantir la fiabilité de la recherche et de la sauvegarde des offres de stages, nous avons mis en place une couverture de tests hybride (Unitaires et Intégration) via **JUnit 5** et **Mockito**. Cette couverture est automatisée via une pipeline CI (GitHub Actions).

- **Tests Unitaires (Business Logic) :**
  - ▶ `SavedJobServiceTest` : Validation des règles de gestion critiques et de la sécurisation des données. Le test vérifie notamment que les URLs d'offres excessivement longues provenant d'API externes sont correctement tronquées à 250 caractères maximum avant l'insertion en base, prévenant ainsi les erreurs critiques de type 500 (`DataTruncationException`).
  - `JobSearchServiceTest` : Validation de la résilience du système. Nous testons le comportement de l'application en cas de défaillance de l'API externe (JSearch down ou timeout) pour s'assurer que l'application ne crashe pas et retourne un état propre (liste vide).
- **Tests d'Intégration (API & Contrôleurs) :**
  - `JobControllerIntegrationTest` : Utilisation de `@WebMvcTest` et `MockMvc` pour valider les endpoints REST.
  - **Forçage Géographique :** Vérification que le contrôleur intercepte la requête utilisateur et ajoute automatiquement la chaîne `"France"` pour éviter les résultats hors-périmètre (ex: offres aux USA).

- **Calcul des KPIs :** Validation de la route `/api/jobs/stats` qui filtre le pipeline de données en base de données pour retourner le décompte exact des offres statuées (`INTERESSE` , `POSTULE` , `REFUSE`), garantissant l'exactitude du Dashboard utilisateur.

4.1.2 Système de Batch Automatisé

Afin de garantir la fiabilité du processus automatisé et d'optimiser l'utilisation de nos quotas d'API, nous avons mis en place une stratégie de validation rigoureuse reposant sur trois niveaux de tests :

- **Test du Contrôleur (`JobPreferenceControllerTest`) :**
  - Utilisation de Mockito pour simuler la base de données.
  - Vérification stricte : si l'étudiant a déjà une alerte, on écrase l'ancienne au lieu d'en recréer une nouvelle (conservation de l'ID).
- **Test Unitaire du Scheduler (`JobBatchSchedulerTest`) :**
  - On isole complètement la logique du batch.
  - On vérifie que la boucle tourne bien sur toutes les préférences et appelle le service de recherche avec les bons paramètres, sans jamais faire de vraie requête HTTP.
- **Test d'Intégration Complet (`BatchIntegrationTest`) :**
  - Le test ultime avec @SpringBootTest et une base de données en mémoire (H2).
  - On simule un compte, on simule une fausse réponse de l'API ("Stage Java"), on lance le Batch manuellement, et on vérifie que la base de données contient bien exactement une nouvelle ligne avec le statut SUGGESTION.
  -

4.2 Tests du module "Agent IA EtudLife"

4.2.1 Tests du Chatbot (ChatService)

Pour garantir la fiabilité du chatbot et la justesse de ses réponses, nous avons mis en place une couverture de tests hybride via JUnit 5 et Mockito.

Tests Unitaires

- `ChatServiceTest` : Validation des règles de détection d'intentions. Nous testons que les salutations (`"bonjour"` , `"salut"` , `"hello"`) et les remerciements (`"merci"` , `"merci beaucoup"`) sont interceptés avant tout appel à Gemini ou au RAG, et retournent une réponse immédiate appropriée. Ces tests vérifient également que la méthode `isJobsIntent()` détecte correctement les intentions liées aux offres sauvegardées à partir de mots-clés variés (`"mes offres"` , `"j'ai postulé"` , `"liste mes candidatures"`), sans faux positifs sur des questions sans rapport.
- `ChatSessionServiceTest` : Validation de la gestion des sessions conversationnelles. On vérifie que l'historique est bien conservé dans Redis, que les messages sont correctement appendés dans l'ordre, et que la session expire après 30 minutes d'inactivité conformément aux règles métiers.
- `RecipeChatService` : ajout d'un service dédié à la détection des questions liées aux recettes et à la génération de réponses dynamiques via l'API Gemini.

- Intégration dans ChatService : ajout d'une nouvelle branche de traitement interceptant les questions liées à la cuisine avant le RAG et déléguant la génération de la réponse à RecipeChatService.

Tests de Non-Régression

- Vérification que l'ajout de nouvelles intentions (offres postulées, offres intéressantes) n'a pas cassé les flux existants (RAG documentaire, appel Gemini). On s'assure que les questions relatives au règlement intérieur ou aux annonces du site continuent de transiter par le bon pipeline sans être interceptées par le filtre emploi.

Tests Fonctionnels (Scénarios End-to-End)

Scénario	Question envoyée	Vérification
Offres intéressantes	`"montre mes offres intéressantes"`	`isJobsIntent()` retourne `true` , `SavedJobService` appelé avec `INTERESSE` , réponse encodée en `JOB_TITLE` / `JOB_ITEM` , **aucun appel Gemini**
Offres postulées	`"liste mes candidatures"`	Même vérification avec le statut `POSTULE`
Aucune offre	Base vide simulée	Retourne `"Vous n'avez postulé à aucune offre pour le moment."` sans erreur
Fallback RAG	Question hors-périmètre	Retourne `"Désolé, je n'ai pas d'information sur ce sujet."` sans hallucination Gemini

Tests d'Interface (Rendu Frontend)

Validation que le format structuré `JOB_ITEM:titre|localisation|lien` est correctement interprété par le frontend pour générer des cartes cliquables. Vérification des cas limites :

Cas limite	Comportement attendu
Titre absent	Affiché `"Offre sans titre"`
Localisation vide	Affiché `"Localisation non précisée"`
Lien manquant	Carte affichée sans bouton de candidature

Tests d’Intégration (Chatbot API)

Afin de valider le bon fonctionnement des endpoints REST du chatbot, nous avons mis en place deux tests d’intégration basés sur `@SpringBootTest` et `WebTestClient` .

Ces tests permettent de vérifier le comportement global de l’API (sessions, messages, historique, streaming) dans un environnement proche de l’exécution réelle.

ChatbotIntegrationTest

Fonction testée	Endpoint	Vérification principale
Création de session	`/api/chat/new-session`	Retour d’un `sessionId`
Validation entrée	`/api/chat/message`	Erreur `400` si question absente

Fonction testée	Endpoint	Vérification principale
Message "bonjour"	`/api/chat/message`	Réponse cohérente + succès
Historique	`/api/chat/history`	Messages correctement retournés
Fermeture session	`/api/chat/close`	Session supprimée
Streaming SSE	`/api/chat/stream`	Envoi des chunks + `[DONE]`

Ce test valide les endpoints principaux et le contrat API backend.

ChatbotConversationIntegrationTest

Fonction testée	Vérification
Session persistante	Même `sessionId` entre plusieurs messages
Enregistrement messages	Messages utilisateur + assistant stockés
Historique complet	Conversation complète récupérée

Ce test valide la continuité conversationnelle du chatbot.

ChatbotRecipeIntegrationTest

Fonction testée	Vérification
Question recette budget	Réponse générée via `RecipeChatService` avec source `"recipes-ai"`
Question recette ingrédients	Réponse cohérente basée sur les ingrédients fournis
Non régression	Les questions classiques (ex: "bonjour") conservent le comportement initial

Ce test valide l'intégration complète de la fonctionnalité recettes dans l'API chatbot.

Choix techniques

Les dépendances externes sont mockées (`GeminiClient` , `PdfKnowledgeBase` , etc.) afin de garantir des tests :

- indépendants des services externes
- rapides et stables
- compatibles avec la CI/CD

5. Matrice de Responsabilités & Réalisations

Fonctionnalité	Lyna Baouche	Alicya-Pearl Marras	Kenza Menad	Dyhia Sellah
Architecture Backend et conteneurisation	✅	✅	✅	✅
Gestion BDD	☐	✅	☐	☐
Gestion des Releases & CI/CD	✅	☐	☐	☐
Documentation & UML	✅	✅	✅	✅
Organisation & Pilotage Agile	✅	✅	✅	✅
Chasseur de stages - Moteur de recherche d'offres via appel API (Sans automatisation)	✅	☐	☐	☐
Chasseur de stages - Sauvegarde des offres (SavedJobs)	✅	☐	☐	☐
Chasseur de stages - Sauvegarde des préférences	☐	✅	☐	☐
Chasseur de stages - Mise en place du Batch	☐	✅	☐	☐
Agent IA-Chatbot(Réglement intérieur, Charte, Examnes)	☐	☐	✅	☐
Agent IA-Chatbot(Focntionnalités de site)	☐	☐	☐	✅
Agent IA-Chatbot(Recettes générées par Gemini (Sans RAG))	☐	☐	☐	✅
Agent IA -Chatbot(offres de stages(postulées et intéressées))	☐	☐	✅	☐
Tests Postman	✅	✅	✅	✅
Tests intégration	✅	✅	✅	✅
Tests unitaires	✅	✅	✅	✅
Tests focntionnels	✅	✅	✅	✅
Tests non regréssion	✅	✅	✅	✅

5. Guide d'Installation & Déploiement

Prérequis

- **Docker et Docker Compose** installés sur la machine cible (recommandé pour l'architecture DevOps complète).
- **Java 17 ou 21** (si exécution en local sans Docker).
- Un fichier `.env` contenant les clés d'API nécessaires (`RAPIDAPI_KEY` , `DB_PASSWORD` , etc.).

Commandes de lancement (Docker - Recommandé)

Pour lancer l'infrastructure complète (Base de données + Backend Spring Boot + Frontend Nginx) :

```
# 1. Cloner le projet et se rendre à la racine
git clone [https://github.com/LynaBaouche/DevOps.git](https://github.com/LynaBaouche/DevOps.git)
cd DevOps

# 2. Lancer les conteneurs en tâche de fond
docker-compose up --build -d

# 3. Vérifier les logs du backend
docker logs -f backend
```